

Apache Iceberg, Explained

Explainer for the Day 2 (Tue Oct 6) 13:45–14:00 block, right after the [Iceberg lab](#). Fifteen minutes on what Iceberg actually is, the three ways BigQuery touches it, and why it's the foundation of your Bronze layer.

What Apache Iceberg is

Apache Iceberg is an **open table format**: a metadata layer that sits over ordinary Parquet (or ORC) files in object storage, like your Cloud Storage buckets. The data files are plain files you can list in the bucket browser. Iceberg adds manifests and snapshots on top of them so that any engine can answer the question "which files make up this table, right now?" Crucially, nobody owns the format — it's an Apache open-source project, and the same Iceberg table can be read by Spark, Trino, Flink, Presto, Hive, and BigQuery at the same time.

That metadata layer buys you the table behaviors you're used to from SQL Server, but over files in a lake: **ACID transactions** (concurrent readers and writers can't corrupt each other; UPDATE/DELETE/MERGE work), **schema evolution** (add, drop, and rename columns without rewriting the table — no "zombie" data), **time travel and rollback** (query the table as of an earlier snapshot, or reset it to a good state), and **hidden partitioning** (partition values are tracked in metadata rather than encoded in folder names, so pruning works without you remembering to filter on the folder convention).

Why does this exist? Plain files in a data lake have no transaction log — two engines writing at once is how tables get corrupted — and Hive-style partitioning forces you to bake partition values into folder paths and remember to filter them by hand (you saw that convention, `order_date=YYYY-MM-DD/`, in this morning's §30 external-tables activity). Iceberg moves both concerns into the metadata. That is what makes a **lakehouse** possible: warehouse-grade tables on lake-grade storage.

The three ways BigQuery touches Iceberg

One line to keep: **'Iceberg' names the file format, not the product. The question that matters is: who can write, and who keeps the metadata?** All three flavors below store Parquet in *your* bucket; they differ in who owns the table metadata and who may write.

Flavor	Who keeps the metadata	Can BigQuery write?	Other engines
Lakehouse table over an existing Iceberg table (what you built in the lab; the BigLake external-	Whoever wrote the Iceberg table (e.g., a Spark job)	No — BigQuery reads in place	The engines that own the table read and write it

Flavor	Who keeps the metadata	Can BigQuery write?	Other engines
table family)			
BigQuery-managed Iceberg tables ("Iceberg managed tables"; formerly called BigLake tables for Apache Iceberg)	BigQuery's catalog	Yes — full DML, time travel, schema evolution; BigQuery writes Parquet into <i>your</i> bucket	Read access via exported Iceberg metadata snapshots / the Storage API
Open-engine access (metadata export / Iceberg REST catalog)	An open catalog	Shares rather than owns	Spark, Trino, and other Iceberg-speaking engines query the same files directly

Verify current state — this area evolves quickly. Product names, preview status, and exactly which engine can write what are among the fastest-moving details in Google Cloud (the managed-table name above has already changed once). Treat this table as the shape of the landscape, and re-check the docs linked below before you commit a production design to a specific flavor.

Why it matters for your Bronze layer

- **Open format = no lock-in.** Your M&O team will ask "will BigQuery hold our data hostage?" — a fair question. The Iceberg answer: the data lives in *your* buckets as open Parquet plus open Iceberg metadata. If you ever leave, you detach the metadata and take the files; nothing was ever inside a proprietary store. And if a second engine (Spark, Trino) ever enters your estate, it reads the same files — no export pipeline, no second copy.
- **Time travel replaces a restore cycle.** Re-running a bad transformation against "yesterday's Bronze" in SQL Server means a backup/restore cycle. With Iceberg snapshots it's a query clause (FOR SYSTEM_TIME AS OF) or a rollback. For a team that replays Dataform releases, this is an operational superpower.
- **Governance arrives early.** BigLake tables over files in GCS support column-level security, row access policies, and dynamic data masking — meaning your Bronze can carry PII columns that analysts physically cannot select, before Dataform ever transforms a row. That is the M10 governance story arriving a zone early.

The ladder you climbed today

Three blocks, three rungs, one ladder — and the rungs are the Bronze section of your architecture doc:

1. **This morning (DIN §30):** files in a bucket became a read-only external table — a metadata pointer.
2. **The lab (13:00):** similar files got ACID table semantics through Iceberg metadata.
3. **The governance angle (M10, 14:00):** the same pattern gains fine-grained security and catalog metadata via BigLake.

For your design, the default recipe is: **land files on GCS → expose them as BigLake/Iceberg tables for queryable, governed Bronze → Dataform builds managed Silver and Gold tables on top**. Reach for BigQuery-*managed* Iceberg tables specifically when non-BigQuery engines must also write the same tables, or when your exit-the-platform story needs open metadata, not just open files. The 3-day Data Warehousing course goes deeper on that trade.

What this replaces in your current stack

Backup/restore cycles for replay and audit scenarios; staging copies made "just to be safe"; and the quiet lock-in of a proprietary storage engine — none of which survive contact with an open table format over files you own.

Test yourself

- ☐ I can explain Iceberg in one sentence: an open metadata layer that gives Parquet files in a bucket table behavior.
- ☐ I can name the three ways BigQuery touches Iceberg and say who owns the metadata in each.
- ☐ I can explain why hidden partitioning beats folder-name partitioning.
- ☐ I can explain to our M&O team why "open format" answers the lock-in question.

Dig deeper

- [Apache Iceberg project site](#) — the format itself, engine-neutral
- [Apache Iceberg managed tables in BigQuery](#)
- [Introduction to BigLake tables](#)

Part of the Iowa DOM DoIT Data Engineering student pack — see ../00_schedule.md.